
maps

Release 1.0.0

AE,SB,BB

Jun 01, 2026

Contents:

1	clfuncs module	1
2	correlate module	4

CLFUNCS MODULE

`clfuncs.correlate_fast(GV, ni, corr)`

Helper function (**numba jit accelerated**) which performs the **Cross TGE** estimator.

Parameters

- **GV** (*np.ndarray*) – Convolved gridded visibility array (3D array) $\mathcal{V}_{cg}(\nu_n)^{XX/YY}$. **GV** structure is (Polarization, Grid points, Frequency).
- **ni** (*np.ndarray*) – 1D array containing the info about which grid point falls into which annular bin. Must contain values in between 0, Nbin – 1.
- **corr** (*np.ndarray*) – 3D array of shape (Nbin, NC, NC).

Returns

This function does not return anything.

Return type

None

`clfuncs.correlate(GV, ni, Nbin)`

Given the gridded visibility data, it performs correlation using **Cross TGE** estimator which is defined as follows :

$$\text{corr}(\nu_a, \nu_b) = \text{Re} [\mathcal{V}_{cg}^{XX}(\nu_a) \mathcal{V}_{cg}^{*YY}(\nu_b) + \mathcal{V}_{cg}^{YY}(\nu_a) \mathcal{V}_{cg}^{*XX}(\nu_b)]$$

Parameters

- **GV** (*np.ndarray*) – Convolved gridded visibility array (3D array) $\mathcal{V}_{cg}(\nu_n)^{XX/YY}$. **GV** structure is (Polarization, Grid points, Frequency).
- **ni** (*np.ndarray*) – 1D array containing the info about which grid point falls into which annular bin. Must contain values in between 0, Nbin – 1.
- **Nbin** (*int*) – No of annular bins the whole *uv* plane has been divided into.

Returns

corr – 3D array of shape (Nbin, NC, NC).

Return type

np.ndarray

Examples

```
>>> import numpy as np
>>> import myutils.clfuncs.clfuncs as cfunc
Imported myutils
Imported clfuncs,   Import Time : 30/05/26 | 12:57:37 PM
>>> # Load binning info
>>> BIN = np.load('/home/cts23ph/git_package_myutils/tge/bin_info_1000007200.npz')
>>> Nbin = int(BIN['Nbin'])      # No of annular bins the whole uv plane has been
↳divided into
>>> ni = BIN['ni']              # Contains info about which grid point fall into
↳which bin
>>> mask = BIN['NI'] >= 0      # mask array
>>> GV = np.load('/home/cts23ph/git_package_myutils/tge/GV_7200.npy')
>>> print(f'GV.shape = {GV.shape}') # GV shape [nstokes,ni,NC] with nstokes = 2
GV.shape = (2, 457, 457, 768)
>>> print(f'No of Bins = {Nbin}')
No of Bins = 20
>>> # perform correlation
>>> # corr shape (Nbin, NC, NC)
>>> corr = cfunc.correlate(GV[:, mask], ni, Nbin)
Channels : 768
Time Taken : 6.861 seconds.
>>> print(f'corr.shape = {corr.shape}')
corr.shape = (20, 768, 768)
```

`clfuncs.cl_dnu_nubar(maps)`

Given the *Multi-Angular Power Spectrum (MAPS)* $C_\ell(\nu_a, \nu_b)$ array, it computes $C_\ell(\bar{\nu}, \Delta\nu)$.

Parameters

`maps` (`np.ndarray`) – MAPS $C_\ell(\nu_a, \nu_b)$ array. Must be shape of (Nbin, NC, NC).

Returns

`maps1` – MAPS $C_\ell(\bar{\nu}, \Delta\nu)$ array having shape (Nbin, NC, 2NC -1).

Return type

`np.ndarray`

Examples

```
>>> maps = np.load('cl_7200.npy')
>>> # shape (Nbin, NC, NC)
>>> print(f'maps.shape : {maps.shape}')
maps.shape : (20, 768, 768)
>>> maps1 = cfunc.cl_dnu_nubar(maps)
>>> # shape (Nbin, NC, 2*NC -1)
>>> print(f'maps1.shape : {maps1.shape}')
maps1.shape : (20, 768, 1535)
```

`clfuncs.cl_dnu(maps1)`

Given the $C_\ell(\bar{\nu}, \Delta\nu)$ array, it computes $C_\ell(\Delta\nu)$ by averaging over $\bar{\nu}$ axis.

Parameters

`maps1` (*np.ndarray*) – $C_\ell(\bar{\nu}, \Delta\nu)$ array. Must be shape of (Nbin, NC, 2NC -1).

Returns

$C_\ell(\Delta\nu)$ array having shape (Nbin, NC).

Return type

np.ndarray

Examples

```
>>> maps2 = cfunc.cl_dnu(maps1)
>>> # shape (Nbin, NC)
>>> print(f'maps2.shape : {maps2.shape}')
maps2.shape : (20, 768)
```

`clfuncs.cl_dnu_nua_nub(maps)`

Given the *Multi-Angular Power Spectrum (MAPS)* $C_\ell(\nu_a, \nu_b)$ array, it computes $C_\ell(\Delta\nu)$. Internally it computes $C_\ell(\bar{\nu}, \Delta\nu)$ first and then using this, it computes $C_\ell(\Delta\nu)$.

Parameters

`maps` (*np.ndarray*) – MAPS $C_\ell(\nu_a, \nu_b)$ array. Must be shape of (Nbin, NC, NC).

Returns

$C_\ell(\Delta\nu)$ array having shape (Nbin, NC).

Return type

np.ndarray

Examples

```
>>> maps2 = cfunc.cl_dnu_nua_nub(maps)
>>> # shape (Nbin, NC)
>>> print(f'maps2.shape : {maps2.shape}')
maps2.shape : (20, 768)
```

CORRELATE MODULE

`correlate.correlate_fast(GV, ni, corr)`

Helper function (**numba jit accelerated**) which performs the **Cross TGE** estimator.

Parameters

- **GV** (*np.ndarray*) – Convolved gridded visibility array (3D array) $\mathcal{V}_{cg}(\nu_n)^{XX/YY}$. **GV** structure is (Polarization, Grid points, Frequency).
- **ni** (*np.ndarray*) – 1D array containing the info about which grid point falls into which annular bin. Must contain values in between 0, Nbin – 1.
- **corr** (*np.ndarray*) – 2D array of shape (Nbin, NC).

Returns

This function does not return anything.

Return type

None

`correlate.correlate(GV, ni, Nbin)`

Given the gridded visibility data, it performs correlation using **Cross TGE** estimator which is defined as follows :

$$\text{corr}(\nu_a, \nu_b) = \text{Re} [\mathcal{V}_{cg}^{XX}(\nu_a) \mathcal{V}_{cg}^{*YY}(\nu_b) + \mathcal{V}_{cg}^{YY}(\nu_a) \mathcal{V}_{cg}^{*XX}(\nu_b)]$$

Then it collapses the frequency axis to get $\text{corr}(\Delta\nu)$.

Parameters

- **GV** (*np.ndarray*) – Convolved gridded visibility array (3D array) $\mathcal{V}_{cg}(\nu_n)^{XX/YY}$. **GV** structure is (Polarization, Grid points, Frequency).
- **ni** (*np.ndarray*) – 1D array containing the info about which grid point falls into which annular bin. Must contain values in between 0, Nbin – 1.
- **Nbin** (*int*) – No of annular bins the whole *uv* plane has been divided into.

Returns

corr – 2D array of shape (Nbin, NC).

Return type

np.ndarray

Examples

```
>>> import numpy as np
>>> import myutils.clfuns.correlate as corrf
Imported myutils
Imported correlate, Import Time : 30/05/26 | 12:45:40 PM
>>> # Load binning info
>>> BIN = np.load('/home/cts23ph/git_package_myutils/tge/bin_info_1000007200.npz')
>>> Nbin = int(BIN['Nbin'])      # No of annular bins the whole uv plane has been
↳ divided into
>>> ni = BIN['ni']              # Contains info about which grid point fall into
↳ which bin
>>> mask = BIN['NI'] >= 0       # mask array
>>> GV = np.load('/home/cts23ph/git_package_myutils/tge/GV_7200.npy')
```

```
>>> print(f'{GV.shape = }')      # GV shape [nstokes,ni,NC] with nstokes = 2
GV.shape = (2, 457, 457, 768)
>>> print(f'No of Bins = {Nbin}')
No of Bins = 20
>>> # perform correlation
>>> # corr shape (Nbin, NC)
>>> corr = corrf.correlate(GV[:, mask], ni, Nbin)
Channels : 768
Time Taken : 5.962 seconds.
>>> print(f'{corr.shape = }')
corr.shape = (20, 768)
```